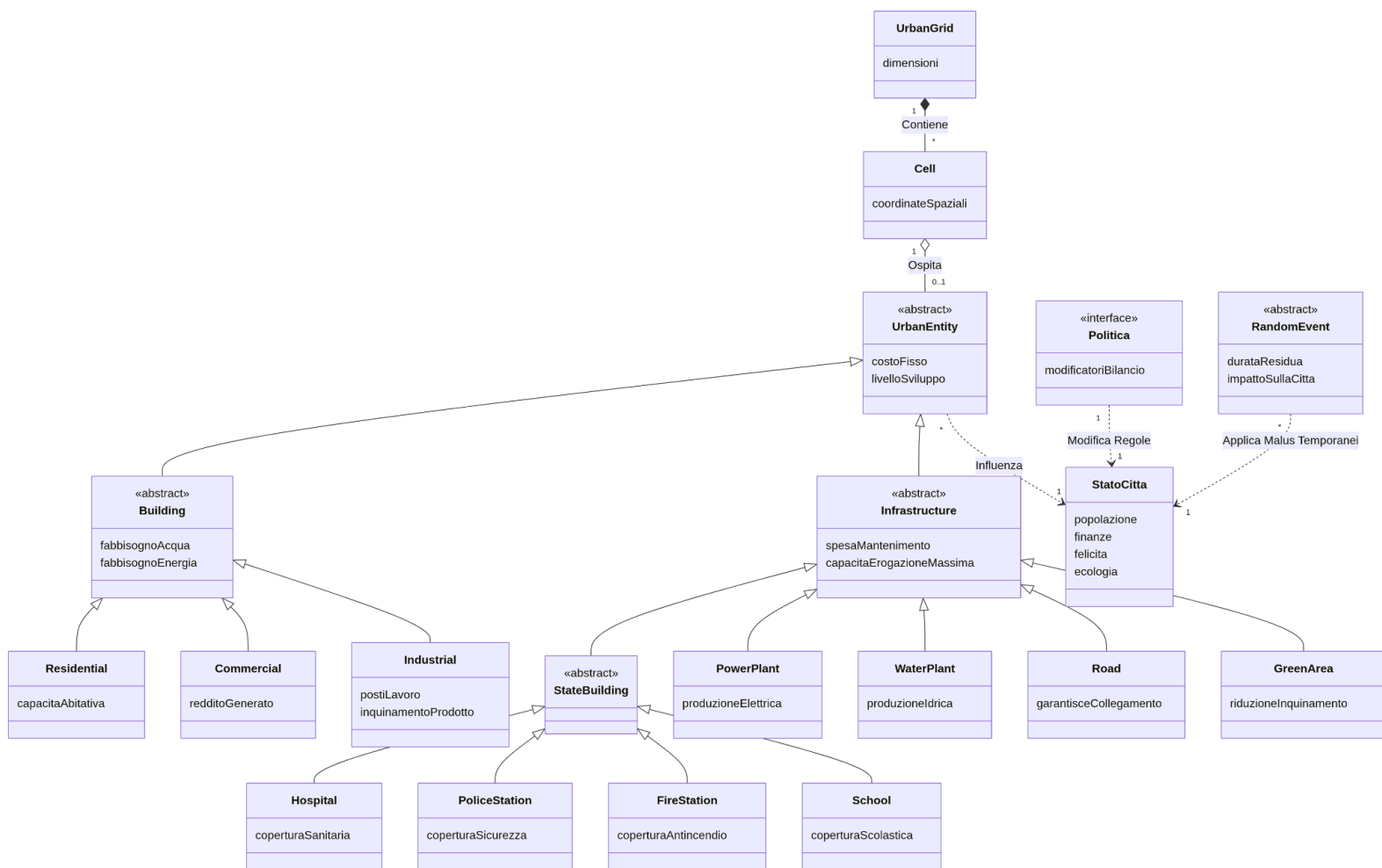


Documento di Design - Progetto SimCity

Domain Model

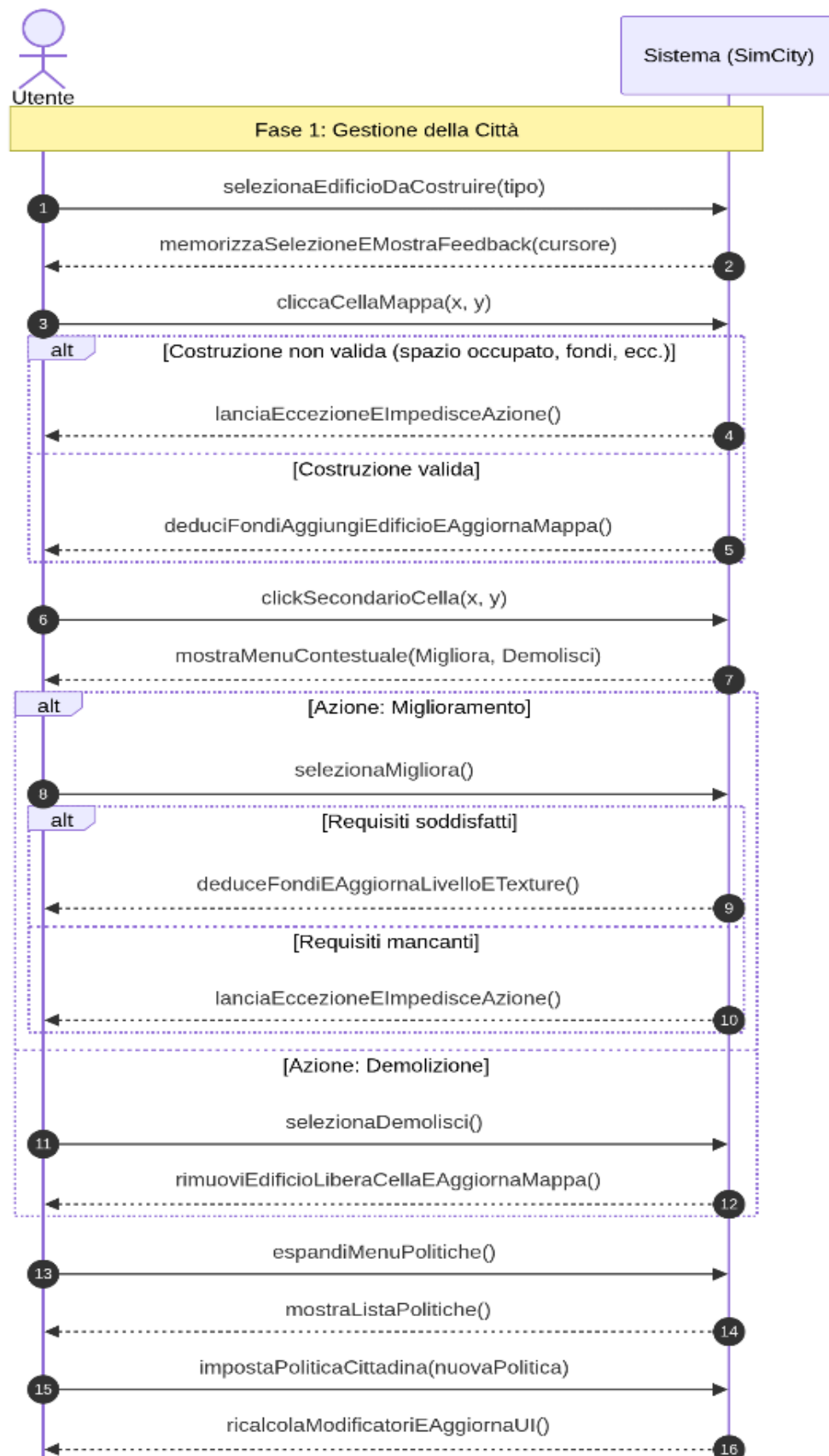


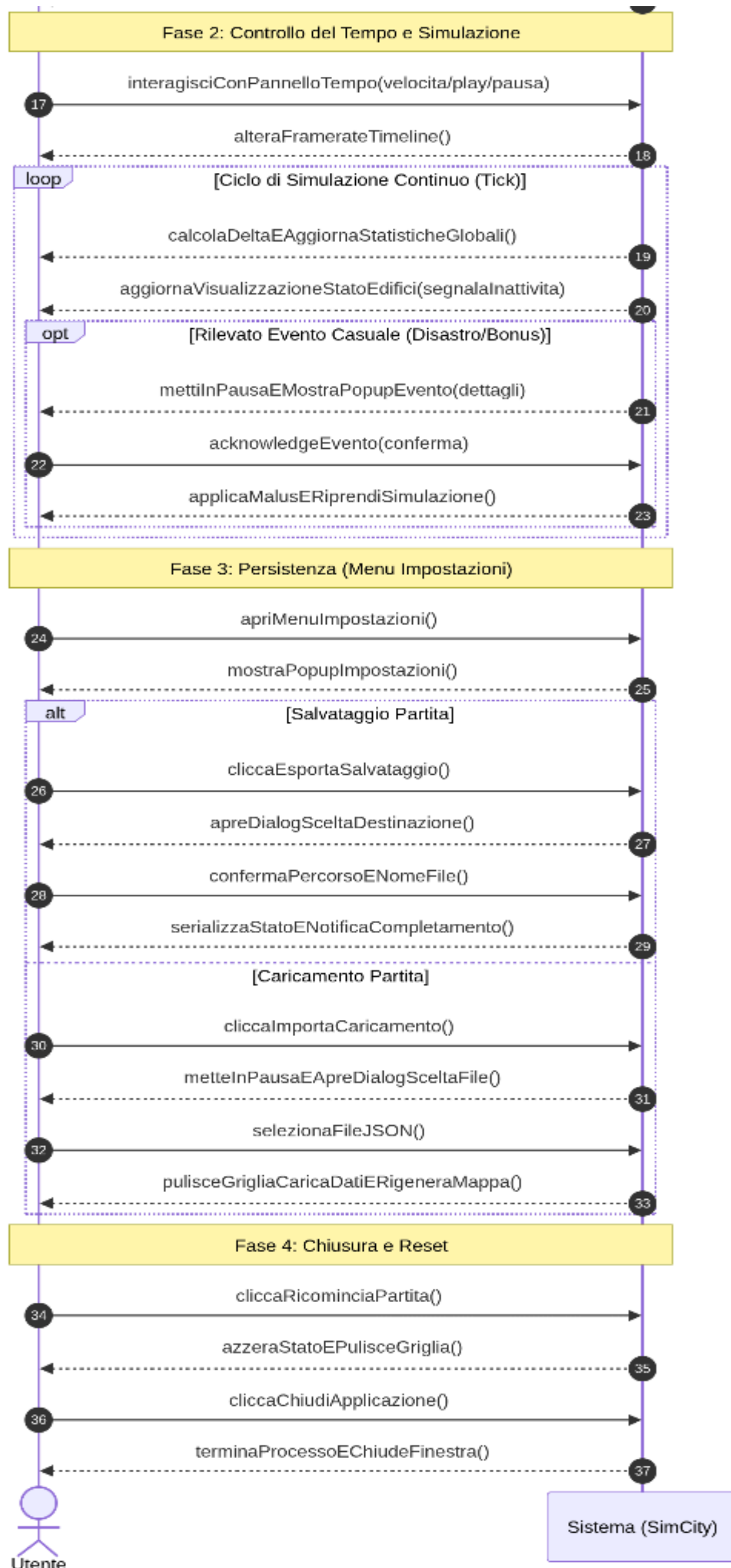
Glossario del dominio

- **StatoCitta:** Custodisce lo stato della città attraverso parametri macroeconomici e metriche
- **UrbanGrid:** È la rappresentazione logica e spaziale della mappa cittadina. Stabilisce i confini e agisce da contenitore
- **Cell:** È l'unità di misura atomica dello spazio. Un singolo lotto con precise coordinate, che può essere vuoto o pieno
- **UrbanEntity:** È l'astrazione di qualsiasi costruzione.
- **Building:** Strutture che consumano costantemente le risorse pubbliche (luce e acqua) e influiscono su popolazione e finanze

- **Infrastructure:** Opere il cui scopo è produrre le risorse essenziali per far funzionare gli altri edifici privato
- **StateBuilding:** Un sotto-tipo di infrastruttura che non produce risorse, ma serve influisce sulle metriche e fornisce i requisiti di base per il funzionamento di Building
- **PoliticaStrategy:** Rappresenta una regola che modifica il calcolo delle metriche
- **RandomEvent:** Evento speciale altera il normale flusso della simulazione

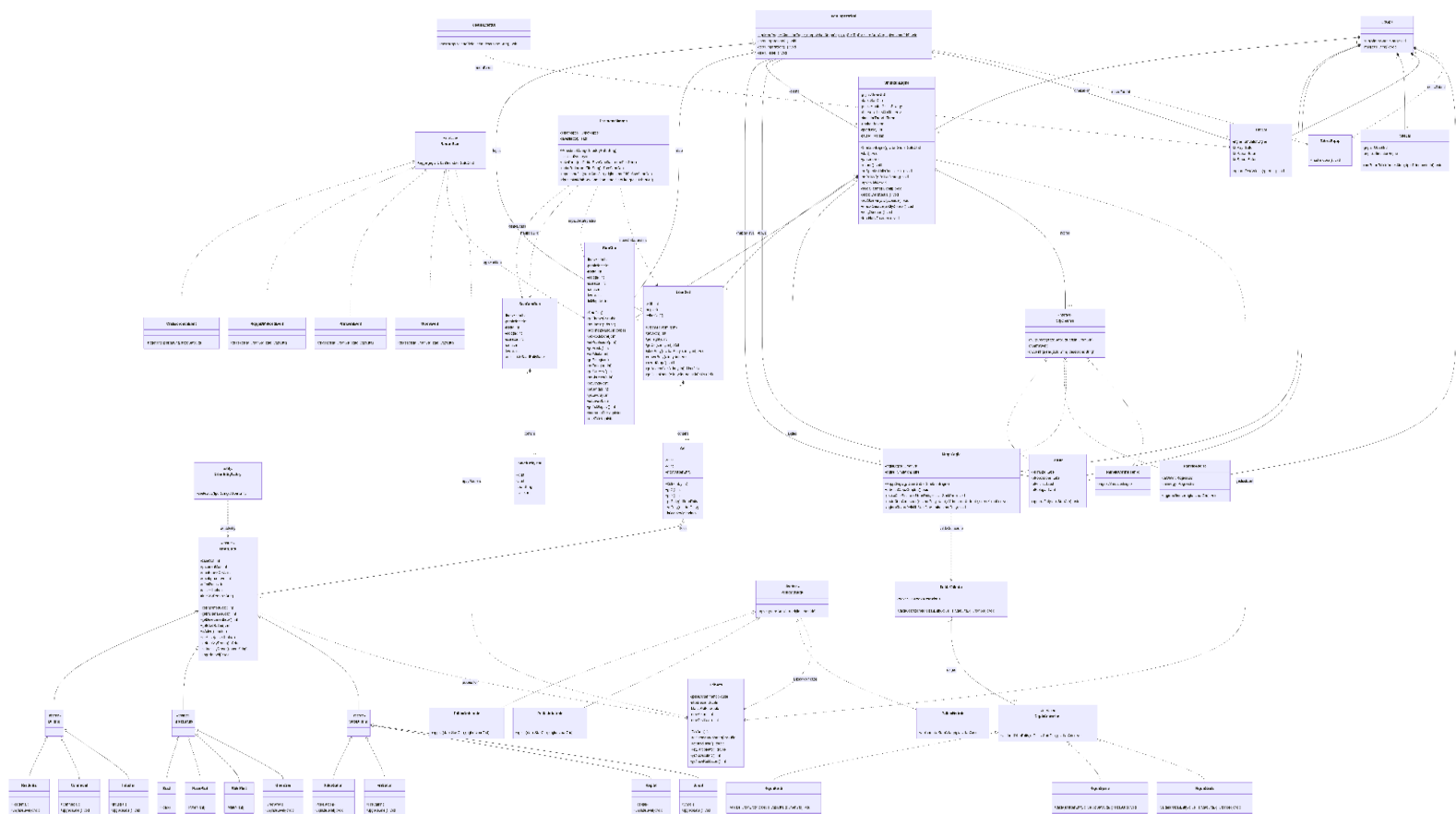
System Sequence diagram





Nella prima fase, le richieste di alterazione della mappa (costruzione, miglioramento o demolizione di entità) vengono sottoposte a una validazione bloccante prima di produrre effetti sullo stato globale della città. Le interazioni specifiche con le entità già posizionate sono centralizzate tramite interfacce contestuali, garantendo un flusso di input standardizzato. L'emanazione di nuove politiche innesca un ricalcolo sincrono dei modificatori, con conseguente aggiornamento immediato della View.

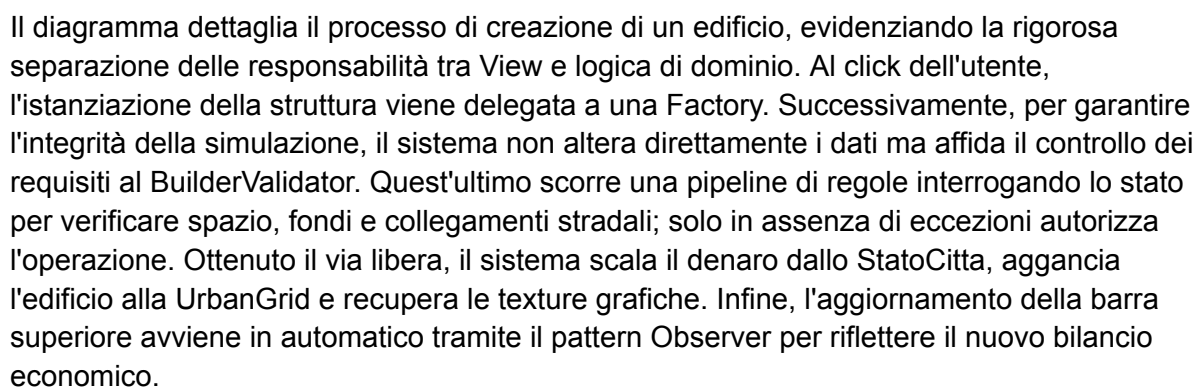
Design Class Model



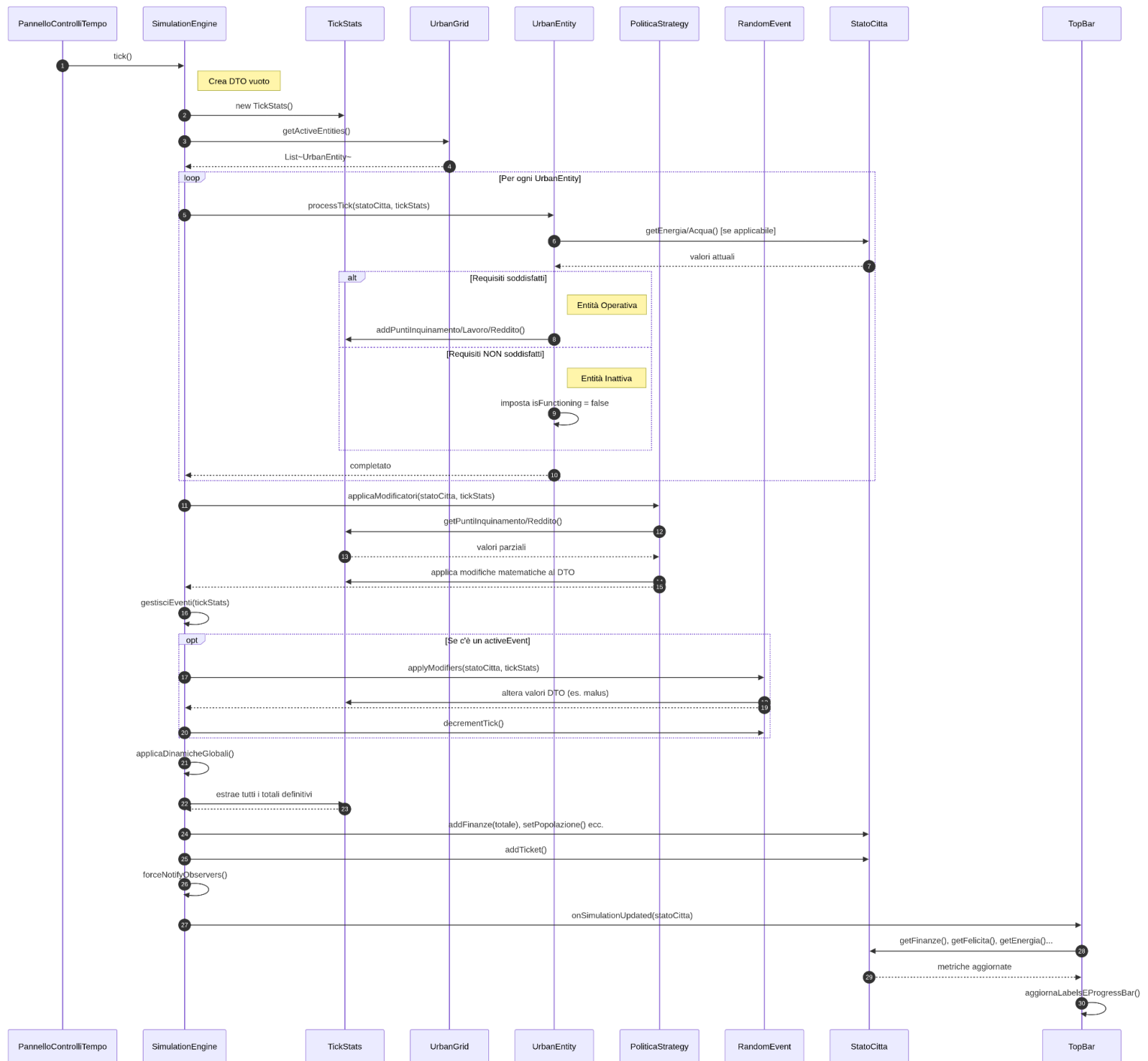
Scelte di design

- Separazione per Livelli (MVC): Il codice è diviso in Domain (Dati e Regole), Core (Motore), UI (Grafica) e Infrastructure (Salvataggi). Ogni pacchetto è isolato per mantenere il codice ordinato.
- Factory Pattern (UrbanEntityFactory): Per creare un nuovo edificio sulla mappa usiamo una Factory. Il motore non usa mai la parola chiave new direttamente, permettendo di istanziare edifici dinamicamente solo tramite il loro "nome".
- Observer Pattern: L'interfaccia utente (UI) si aggiorna automaticamente ad ogni "tick" di gioco mettendosi in ascolto del SimulationEngine tramite l'interfaccia CityObserver.
- Strategy Pattern (Politiche): Le diverse politiche cittadine (Industriale, Ambientale, ecc.) sono implementazioni di un'interfaccia PoliticaStrategy. Cambiare politica significa semplicemente scambiare l'oggetto strategia in esecuzione, senza riempire il motore di complessi cicli if/else.
- Single Responsibility (Persistenza): Il SimulationEngine fa solo il gioco. Il compito di leggere e scrivere file JSON è delegato interamente a un PersistenceManager dedicato, che lavora usando oggetti leggeri di trasferimento dati (DTO).
- Separazione per Livelli (MVC): Il codice è diviso in Domain (Dati e Regole), Core (Motore), UI (Grafica) e Infrastructure (Salvataggi). Ogni pacchetto è isolato per mantenere il codice ordinato.

Costruzione edificio



Avanzamento tick



A intervalli regolari, l'Engine esegue un tick() per far avanzare la simulazione. Il diagramma evidenzia l'uso combinato di molteplici design pattern:

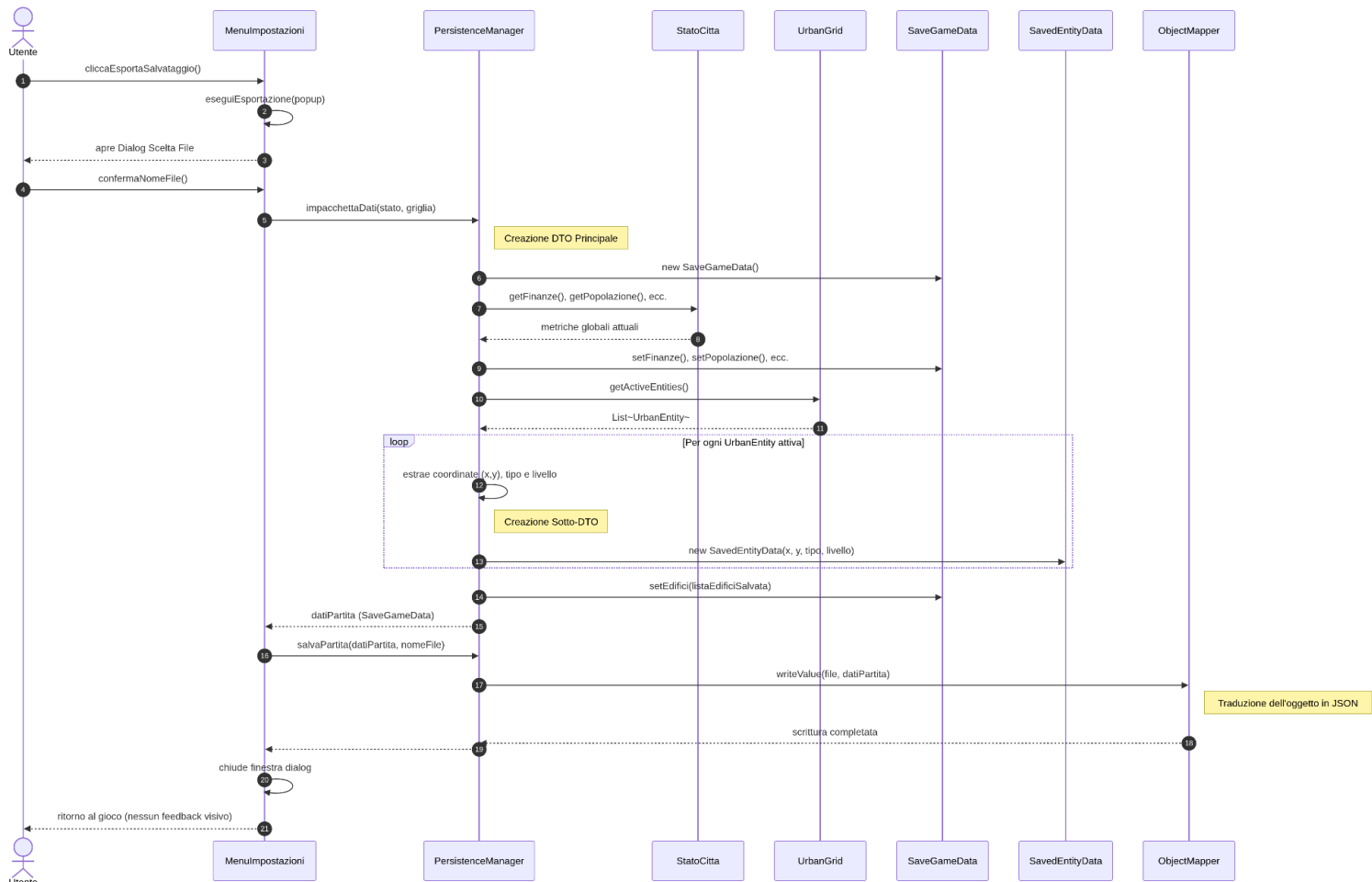
Data Transfer Object (DTO): Il motore istanzia un oggetto temporaneo (TickStats) per raccogliere tutti i calcoli parziali del turno.

Polimorfismo e Delegation: Il motore itera sugli edifici attivi chiamando processTick(). Ogni entità verifica indipendentemente i propri requisiti e versa i propri bonus o malus (soldi, inquinamento, energia) all'interno del DTO temporaneo.

Strategy: L'Engine applica la PoliticaStrategy (e gli eventuali RandomEvent) attualmente in vigore. Questi leggono il DTO e ne alterano i parametri matematicamente, eliminando la necessità di logica o blocchi if-else hardcoded all'interno del motore.

Observer: Al termine del ciclo, dopo aver riversato i dati finali nello stato della città, il motore invoca forceNotifyObservers(). Le interfacce UI ricevono i nuovi dati e ridisegnano le barre e i testi in modo totalmente disaccoppiato.

Salvataggio partita



Il diagramma descrive il disaccoppiamento tra il gioco in esecuzione e il file system. Durante il salvataggio la UI chiama direttamente il **PersistenceManager**, il quale impacchetta le coordinate e le statistiche in un DTO (Data Transfer Object) chiamato `SaveGameData` e lo serializza in JSON.